

DIA 2 — Como Fazer Seu Banco de Dados Performar Melhor

Quando pensamos em performance de banco de dados, é comum imaginar consultas SQL complexas, índices ou otimizações de código. Embora todos esses fatores sejam importantes, existe uma etapa anterior que muitas vezes é negligenciada: a configuração do próprio Sistema Gerenciador de Banco de Dados (SGBD).

Um banco de dados mal configurado pode desperdiçar memória, utilizar o disco de forma excessiva e comprometer significativamente o desempenho da aplicação, independentemente da qualidade das consultas SQL.

Por esse motivo, antes de estudar técnicas avançadas de otimização, é fundamental compreender alguns dos principais parâmetros responsáveis pela utilização da memória no PostgreSQL.

Neste capítulo veremos três configurações extremamente importantes: **shared_buffers**, **work_mem** e **maintenance_work_mem**.

Shared Buffers

O parâmetro **shared_buffers** define a quantidade de memória RAM que o PostgreSQL utilizará como cache interno para armazenar páginas de dados acessadas com frequência.

Sempre que uma consulta precisa ler informações de uma tabela, o banco tenta encontrá-las primeiro nessa área de memória.

Caso os dados já estejam armazenados no cache, o acesso ocorre praticamente de forma instantânea.

Caso contrário, o PostgreSQL precisa buscá-los no disco, uma operação milhares de vezes mais lenta que um acesso à memória RAM.

Por esse motivo, o **shared_buffers** é um dos parâmetros mais importantes para o desempenho geral do banco.

Na maioria dos cenários, recomenda-se configurar esse parâmetro entre **15% e 25% da memória RAM disponível** no servidor.

Por exemplo:

- Servidor com 16 GB de RAM → aproximadamente 4 GB de `shared_buffers` ;
- Servidor com 32 GB de RAM → aproximadamente 8 GB de `shared_buffers` ;
- Servidor com 64 GB de RAM → aproximadamente 16 GB de `shared_buffers` .

Vale lembrar que não existe um valor universal.

A configuração ideal depende da carga de trabalho da aplicação, da quantidade de memória disponível e do sistema operacional utilizado.

Work Memory (work_mem)

O parâmetro **work_mem** controla a quantidade de memória que cada operação individual poderá utilizar durante a execução de consultas.

Essa memória é utilizada principalmente em operações como:

- Ordenações (`ORDER BY`);
- Agrupamentos (`GROUP BY`);
- Operações de Hash Join;
- Merge Join;
- Criação de índices temporários;
- Outras operações que exigem processamento intermediário.

Uma característica muito importante é que **work_mem não é reservado apenas uma vez para toda a instância do banco**.

Cada operação executada por cada consulta pode consumir até esse limite.

Imagine uma consulta que realiza três ordenações simultaneamente.

Ela poderá utilizar três vezes o valor configurado em `work_mem`.

Se dezenas ou centenas de conexões estiverem executando consultas complexas ao mesmo tempo, o consumo total de memória poderá crescer rapidamente.

Por isso, uma boa prática consiste em calcular aproximadamente:

$\text{work_mem} \approx 25\% \text{ da memória RAM} \div \text{max_connections}$

Por exemplo:

Servidor com:

- 32 GB de RAM;
- 100 conexões simultâneas.

Teríamos aproximadamente:

25% de 32 GB = 8 GB

8 GB $\div$ 100 conexões $\approx$ **80 MB por conexão**.

Esse cálculo serve apenas como ponto de partida.

O valor ideal deve sempre ser ajustado considerando o perfil das consultas executadas pela aplicação.

Maintenance Work Memory (maintenance_work_mem)

Enquanto o **work_mem** é utilizado durante consultas executadas pelos usuários, o parâmetro **maintenance_work_mem** é reservado para tarefas administrativas do banco de dados.

Entre elas podemos destacar:

- Criação de índices (`CREATE INDEX`);
- Reconstrução de índices (`REINDEX`);
- Execução do `VACUUM` ;
- Execução do `VACUUM FULL` ;
- Operações de `ALTER TABLE` ;
- Processos de manutenção realizados pelo autovacuum.

Essas tarefas normalmente não acontecem com tanta frequência quanto as consultas executadas pelas aplicações, mas podem consumir bastante processamento quando ocorrem.

Quanto maior esse parâmetro, mais memória essas operações poderão utilizar e, conseqüentemente, menor tende a ser o tempo necessário para concluí-las.

Como ponto de partida, costuma-se configurar aproximadamente **5% da memória RAM do servidor**.

Por exemplo:

- 16 GB de RAM → cerca de 800 MB;
- 32 GB de RAM → cerca de 1,6 GB;
- 64 GB de RAM → cerca de 3,2 GB.

Assim como os demais parâmetros, esse valor deve ser ajustado conforme a carga de trabalho do ambiente.

Conclusão

Esses três parâmetros representam apenas uma pequena parte das configurações disponíveis no PostgreSQL, mas já exercem um impacto significativo no desempenho da maioria das aplicações.

De forma resumida:

- **shared_buffers** define quanto da memória será utilizado para armazenar páginas de dados em cache, reduzindo acessos ao disco.
- **work_mem** determina a memória disponível para operações executadas durante consultas, como ordenações e agrupamentos.

- **maintenance_work_mem** controla a memória utilizada por tarefas administrativas, como criação de índices e execução do VACUUM.

Compreender esses parâmetros é o primeiro passo para extrair o máximo desempenho do PostgreSQL.

Nos próximos capítulos, veremos que uma boa configuração de memória, aliada a uma modelagem correta, índices bem planejados e consultas otimizadas, permite que um banco de dados processe milhões — e até bilhões — de registros com excelente desempenho.